

Complete Project Documentation and Free Deployment Manual

Version: 2.0

Date: September 19, 2026

Project: Multi-Agent Financial Research System

1. Executive Overview

FinResearch AI is a full-stack financial research workspace for finance students, MBA candidates, researchers, and early-career analysts. It converts annual reports, 10-K filings, earnings transcripts, investor presentations, and other financial documents into searchable evidence, structured metrics, risk indicators, company comparisons, reports, and conversational research answers.

The system is designed around five cooperating agents:

1. Document Agent extracts text, normalizes it, splits it into searchable chunks, detects sections, and indexes the result.
2. Extraction Agent extracts financial metrics such as revenue, margins, income, assets, debt, ratios, EPS, and cash flow.
3. Risk Agent identifies liquidity, debt, regulatory, market, concentration, and operational risks.
4. Benchmark Agent compares companies across metrics and risk profiles.
5. Research Agent answers questions using document evidence, stored metrics, stored risks, and citations.

The application includes a demo dataset for Apple, Microsoft, Tesla, and Amazon. It also supports registration, credential login, research sessions, document upload, dashboards, charts, reports, and API access.

***Important limitation:** financial analysis is informational and educational. It is not investment, legal, accounting, or tax advice. Always verify extracted values against the original filing.*

2. Product Capabilities

2.1 Authentication

- Create an account with name, email, and password.
- Sign in with email and password.
- JWT sessions are managed by NextAuth.
- Passwords are hashed with bcryptjs.
- Protected dashboard APIs require an authenticated session.

2.2 Research Sessions

A session is a workspace for a research project. A session can contain:

- A name and description.
- Tags.
- Uploaded documents.
- Chat messages.
- Generated reports.
- Agent activity.

2.3 Document Upload

Supported formats:

- PDF with selectable text.
- DOCX.
- TXT.

The upload workflow stores the document, extracts content, creates chunks, extracts metrics, scans risks, and marks the document as completed. The database stores a short file type value (pdf, docx, or txt) so long MIME strings do not exceed the database column limit.

Scanned or image-only PDFs require OCR before their text can be analyzed. The application cannot reliably extract financial values from a PDF that contains only page images.

2.4 Financial Metrics

The metric model supports:

- Revenue and revenue growth.
- Gross profit and gross margin.
- Operating income and operating margin.
- Net income and net margin.
- EBITDA and EBITDA margin.
- Total assets, liabilities, equity, cash, and debt.
- Current ratio, quick ratio, debt-to-equity.
- ROE, ROA, EPS, and P/E ratio.
- Operating cash flow, capital expenditures, and free cash flow.
- Raw extracted values and extraction metadata.

2.5 Risk Analysis

Risk records contain:

- Risk type.
- Severity.
- Title.
- Description.
- Source text.
- Page reference when available.
- Recommendation.

If Gemini is unavailable or quota is exhausted, the application uses a local fallback that detects labeled financial values and common risk indicators. Gemini remains the preferred provider when it is available.

2.6 Benchmarking and Reports

Benchmarking compares two or more companies using stored metrics and risks. Reports can contain:

- Executive summary.
- Key findings.
- Metric comparisons.
- Risk summary.
- Recommendations.
- Full report content.

When AI quota is unavailable, the application uses stored data and fallback text where supported. Do not treat fallback text as equivalent to a full AI-generated analyst report.

2.7 Research Agent

The Research Agent searches document chunks, reads stored metric rows and risk rows, and sends a grounded prompt to Gemini. It returns an answer with source citations and reasoning metadata.

When Gemini is unavailable, the Research Agent switches to Local Research Mode and returns:

- The user question.
- Stored metrics.
- Stored risk indicators.
- Relevant document excerpts.
- Citations to document names and sections.

3. Technology Architecture



3.1 Repository Layout

src/app/	Next.js pages and API routes
src/app/api/	Authentication and application APIs
src/app/dashboard/	Dashboard screens
src/db/schema.ts	PostgreSQL schema definitions
src/db/index.ts	Drizzle database connection
src/lib/auth.ts	NextAuth configuration
src/lib/seed.ts	Demo data seeding
src/lib/seedData.ts	Seed companies and documents
src/lib/agents/	Document, extraction, risk, benchmark, research agents
public/	Documentation and static files
package.json	Scripts and dependencies
drizzle.config.json	Drizzle schema and database configuration

Correct file name: drizzle.config.json.

3.2 Agent Pipeline

```
Upload
|
v
Extract PDF/DOCX/TXT text
|
v
Create document database row
|
v
Document Agent: clean, section, chunk, index
|
+--> Extraction Agent: metrics via Gemini or local fallback
|
+--> Risk Agent: risks via Gemini or local fallback
|
v
Mark document completed
```

The document is useful even when Gemini fails. Core indexing must succeed first. Gemini failures should be treated as optional and not as upload failures.

4. Source Code Map

Frontend pages

- / redirects authenticated users to /dashboard and others to /login.
- /login provides credential login.
- /register creates a user account.
- /dashboard shows summary statistics and demo-data loading.
- /dashboard/sessions manages research sessions.
- /dashboard/documents lists and uploads documents.
- /dashboard/documents/[id] shows content, metrics, and risks.
- /dashboard/companies lists companies.
- /dashboard/metrics displays financial charts and tables.
- /dashboard/risks displays risk filters and cards.
- /dashboard/benchmark compares companies.
- /dashboard/reports creates and reads reports.
- /dashboard/research provides the Research Agent chat.
- /dashboard/docs contains in-app documentation.

API route map

- /api/auth/register creates users.
- /api/auth/[...nextauth] handles sign-in, sign-out, and sessions.

- /api/health tests application and database health.
- /api/seed loads demo data.
- /api/sessions manages sessions.
- /api/sessions/[id] reads, updates, and deletes one session.
- /api/documents lists and uploads documents.
- /api/documents/[id] reads or deletes one document.
- /api/companies lists company data.
- /api/metrics reads financial metrics.
- /api/risks reads risks.
- /api/benchmark generates comparisons.
- /api/reports lists and creates reports.
- /api/reports/[id] reads and deletes one report.
- /api/chat reads and creates research chat messages.
- /api/dashboard returns dashboard statistics.

5. Prerequisites

Required

- Windows, macOS, or Linux.
- Node.js 20.9 or newer is recommended for Next.js 16.
- npm.
- PostgreSQL 14 or newer.
- Git.

Optional

- Docker Desktop for PostgreSQL.
- Google AI Studio account and Gemini API key.
- A searchable PDF viewer or OCR application for scanned filings.
- A GitHub account for deployment.

Check versions:

```
node --version
npm --version
psql --version
git --version
```

6. Complete Local Installation on Windows

Step 1: Open the project

```
cd "C:\Users\Dell\Desktop\INFOSYS SPRINGBOARD\multi-agent-financial-research-system"
```

For a cloned project, use its actual folder instead.

Step 2: Install dependencies

```
npm install
```

The project currently pins pdf-parse to 1.1.1 because the v2 package requires a worker that can fail under Next.js Turbopack server.

Step 3: Start PostgreSQL

Option A: PostgreSQL Windows service

Confirm the service:

```
Get-Service | Where-Object { $_.Name -match "postgres" }
```

Start it if needed:

```
Start-Service postgresql-x64-18
```

The service name may differ by installed version.

Option B: Docker PostgreSQL

```
docker run --name finresearch-postgres `
-e POSTGRES_USER=postgres `
-e POSTGRES_PASSWORD=postgres `
-e POSTGRES_DB=app_db `
-p 5432:5432 `
-d postgres:16
```

If the container already exists:

```
docker start finresearch-postgres
```

Step 4: Create the database

With PostgreSQL credentials configured:

```
psql -U postgres -h 127.0.0.1 -p 5432
```

Run:

```
CREATE DATABASE app_db;
\q
```

If the database already exists, continue.

Step 5: Configure `.env`

Create `.env` in the project root. Never commit this file.

```
DATABASE_URL=postgresql://postgres:YOUR_POSTGRES_PASSWORD@127.0.0.1:5432/app_db
NEXTAUTH_SECRET=replace-with-a-long-random-secret
NEXTAUTH_URL=http://localhost:3000
GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.6-flash
GEMINI_PRO_MODEL=gemini-3.6-flash
```

Replace `YOUR_POSTGRES_PASSWORD` with the real PostgreSQL password. If the password is `postgres`, use:

```
DATABASE_URL=postgresql://postgres:postgres@127.0.0.1:5432/app_db
```

Generate a NextAuth secret in PowerShell:

```
$bytes = New-Object byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
[Convert]::ToBase64String($bytes)
```

Copy the output after `NEXTAUTH_SECRET=`.

A Gemini API key is optional for basic operation because the application has local fallback behavior. Full Gemini analysis requires a valid key with available quota.

Step 6: Apply the database schema

Preferred command:

```
npx drizzle-kit push
```

If the database is empty and push hangs during introspection, generate and apply a migration:

```
npx drizzle-kit generate
npx drizzle-kit migrate
```

The application should have tables including `users`, `documents`, `document_chunks`, `financial_metrics`, and `risk_flags`.

Step 7: Start the application

```
npm run dev
```

Open:

```
http://localhost:3000
```

If port 3000 is occupied, Next.js may use 3001. Use the URL printed in the terminal.

Stop the server with the keyboard shortcut Ctrl+C. Do not type Ctrl+C into PowerShell as text.

Step 8: Check health

In a second PowerShell terminal:

```
Invoke-WebRequest http://localhost:3000/api/health
```

A healthy response includes:

```
{  
  "status": "ok",  
  "database": "connected"  
}
```

Step 9: Seed demo data

```
Invoke-WebRequest -Uri "http://localhost:3000/api/seed" -Method GET
```

Expected response:

```
{"success":true,"message":"Database seeded successfully"}
```

The seed creates demo users, companies, documents, metrics, risks, and a demo session.

Step 10: Sign in

```
Email: demo@finresearch.ai  
Password: demo123456
```

The second demo user is:

```
Email: student@finresearch.ai  
Password: demo123456
```

Step 11: Validate the project

```
npm run typecheck  
npm run lint  
npm run build
```

The npm output lines beginning with > are informational. Never type them manually.

7. Using the Application End to End

Workflow A: Demo dataset

1. Sign in with the demo account.
2. Open Dashboard.
3. Click Load Demo Data if the database is empty.
4. Open Companies to inspect Apple, Microsoft, Tesla, and Amazon.
5. Open Financial Metrics to compare ratios and margins.
6. Open Risk Analysis to filter risks.
7. Open Benchmarking to select companies.
8. Open Reports to generate an analysis.
9. Open Research Agent and select the seeded session.

Workflow B: Upload a searchable PDF

1. Open Documents.
2. Click Upload Document.
3. Enter company name and ticker.
4. Select document type and fiscal year.
5. Select a research session if desired.
6. Choose a PDF with selectable text.
7. Wait for indexing and agent processing.
8. Open the document detail page.
9. Review Raw Content, Metrics, and Risks.

If the raw content is only a fallback message, the PDF parser could not read text. Use an OCR/searchable version of the filing.

Workflow C: Upload DOCX or TXT

1. Use a .docx or .txt file.
2. Complete the same metadata fields.
3. Upload the file.
4. Wait for processing.
5. Review extracted content and analysis.

Workflow D: Ask a research question

Good questions are specific and evidence-based:

- What is the revenue trend and operating margin?
- Which company has the strongest liquidity position?
- What risks are mentioned in the Risk Factors section?

- Compare cash flow and debt-to-equity across two companies.
- What evidence supports the concern about concentration risk?

When Gemini quota is unavailable, Local Research Mode uses stored data and citations. It is still useful for retrieval, but it does not provide full generative reasoning.

8. Environment Variables Reference

Variable	Required	Purpose
DATABASE_URL	Yes	PostgreSQL connection string.
NEXTAUTH_SECRET	Yes	Signs and encrypts authentication session data.
NEXTAUTH_URL	Yes	Canonical application URL.
GEMINI_API_KEY	Optional	Enables Gemini extraction, risk, chat, benchmark, and report generation.
GEMINI_MODEL	Optional	Default model for fast operations.
GEMINI_PRO_MODEL	Optional	Report model setting.

Never place PowerShell commands inside .env. For example, this is wrong:

```
GEMINI_API_KEY= $secret = ...
```

Only values belong in .env.

9. Free Deployment: Vercel + Neon

This is the recommended free deployment route for a Next.js project.

9.1 What you need

- GitHub account.
- Vercel account.
- Neon account.
- Google AI Studio key if full AI features are needed.

Free tiers and limits change over time. Review current provider limits before using the deployment for real users.

9.2 Create a Neon PostgreSQL database

1. Visit <https://neon.tech>.
2. Create an account using GitHub or email.
3. Create a new project.
4. Choose a region near your users.
5. Open the project dashboard.
6. Copy the pooled or standard PostgreSQL connection string.
7. Keep the connection string private.

A Neon URL commonly looks like:

```
postgresql://user:password@host.neon.tech/database?sslmode=require
```

9.3 Prepare the repository

From the project directory:

```
git status
git add .
git commit -m "Prepare FinResearch AI deployment"
```

Create a GitHub repository, then connect and push:

```
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
git branch -M main
git push -u origin main
```

Do not commit .env or API keys.

9.4 Create the Vercel project

1. Visit <https://vercel.com>.
2. Sign in with GitHub.
3. Click Add New Project.
4. Import the GitHub repository.
5. Keep the framework as Next.js.
6. Use the default build command unless Vercel detects otherwise.
7. Add environment variables before deploying.

9.5 Configure Vercel environment variables

Add these variables in Vercel Project Settings, for Production, Preview, and Development as appropriate:

```
DATABASE_URL=your Neon connection string
NEXTAUTH_SECRET=a-new-production-secret
```

```
NEXTAUTH_URL=https://your-app.vercel.app
GEMINI_API_KEY=your-valid-key
GEMINI_MODEL=gemini-3.6-flash
GEMINI_PRO_MODEL=gemini-3.6-flash
```

Use a different NEXTAUTH_SECRET from local development. Do not reuse an exposed secret.

9.6 Deploy

Click Deploy. Vercel runs the Next.js build and provides a URL such as:

```
https://finresearch-ai-yourname.vercel.app
```

Update NEXTAUTH_URL to the exact production URL, then redeploy if necessary.

9.7 Apply the production schema

Use the Neon connection string temporarily in a secure local environment, or set it in a separate shell session. Do not overwrite your local .env permanently.

PowerShell example:

```
$env:DATABASE_URL="YOUR_NEON_CONNECTION_STRING"
npx drizzle-kit push
Remove-Item Env:DATABASE_URL
```

If drizzle-kit push is unreliable:

```
$env:DATABASE_URL="YOUR_NEON_CONNECTION_STRING"
npx drizzle-kit generate
npx drizzle-kit migrate
Remove-Item Env:DATABASE_URL
```

Confirm the production schema before seeding.

9.8 Seed production demo data

After deployment and schema creation:

```
Invoke-WebRequest -Uri "https://your-app.vercel.app/api/seed" -Method GET
```

Then open the production URL and sign in with the demo account. For a real deployment, change or remove demo credentials im

9.9 Verify production

Check:

```
https://your-app.vercel.app/api/health
https://your-app.vercel.app/login
https://your-app.vercel.app/dashboard
```

Test:

- Registration.
- Login and logout.
- Session creation.
- Demo data.
- Document upload.
- Metrics and risks.
- Research Agent.
- Reports.

Inspect Vercel Function Logs if an API route fails.

9.10 Vercel deployment limitations

- Serverless functions have execution and memory limits.
- Large PDF uploads may exceed platform request limits.
- Long-running AI report generation may time out.
- The database must be reachable from Vercel.
- Do not store large binary files directly in PostgreSQL text columns for production scale.
- Use object storage such as Vercel Blob, Cloudflare R2, or Supabase Storage for large files.

10. Other Free or Low-Cost Deployment Options

10.1 Render

1. Create a Render account.
2. Create a free PostgreSQL database if available in your region.
3. Create a Web Service from GitHub.
4. Set build command to `npm run build`.
5. Set start command to `npm run start`.
6. Add the environment variables.
7. Deploy.
8. Apply the schema against the Render database.
9. Seed `/api/seed`.

Render free services may sleep when inactive, so the first request can be slow.

10.2 Railway

1. Create a Railway account.
2. Create a project from GitHub.
3. Add PostgreSQL.

4. Add the Next.js service.
5. Configure environment variables.
6. Use `npm run build` and `npm run start`.
7. Apply the schema using the Railway database URL.
8. Seed the application.

Railway pricing and trial limits change, so verify current terms.

10.3 Fly.io or a small VPS

For a Docker-based deployment:

1. Add a production Dockerfile.
2. Build the image.
3. Push it to a container registry.
4. Run the container with environment variables.
5. Use managed PostgreSQL.
6. Configure HTTPS and backups.

This gives more control but requires more operations work than Vercel.

11. Production Security Checklist

Before sharing the application publicly:

- Rotate every API key exposed during development.
- Use a unique strong `NEXTAUTH_SECRET` in production.
- Use a production-only database.
- Never commit `.env`.
- Remove or protect the `/api/seed` endpoint after seeding.
- Change or remove demo passwords.
- Restrict database network access where possible.
- Add rate limiting for registration, chat, upload, and AI routes.
- Add file size and content validation.
- Use object storage for production documents.
- Configure database backups.
- Monitor provider quotas and billing.
- Review logs for personal or confidential financial documents.
- Do not expose raw database errors to end users.

12. Troubleshooting Guide

``npm run dev`` says port 3000 is in use

Find the process:

```
Get-NetTCPConnection -LocalPort 3000 -State Listen
```

Stop the process by PID:

```
taskkill /PID YOUR_PID /F
```

Start again:

```
npm run dev
```

``>`` is not recognized in PowerShell

Do not type npm output lines. Run only:

```
npm run typecheck
```

PostgreSQL password authentication failed

Your DATABASE_URL password does not match PostgreSQL. Reset it in pgAdmin or SQL Shell:

```
ALTER USER postgres WITH PASSWORD 'new-password';
```

Update .env, then rerun the database command.

``users`` table does not exist

Apply the schema:

```
npx drizzle-kit push
```

Then seed:

```
Invoke-WebRequest http://localhost:3000/api/seed -Method GET
```

Seed returns HTTP 500

Check:

1. PostgreSQL is running.

2. DATABASE_URL is correct.
3. The schema exists.
4. The development server was restarted after .env changes.
5. The terminal running Next.js contains the underlying error.

`value too long for type character varying(50)` during upload

The application stores the short extension (pdf, docx, or txt) in file_type. Restart the server after pulling the fix and upload again.

PDF contains only fallback text

The PDF parser could not extract text. Use a PDF with selectable text. Scanned documents need OCR. Delete the failed record and upload the searchable version again.

PDF parser tries to open `test/data/05-versions-space.pdf`

This indicates the old parser wrapper or stale Next.js process is running. Ensure pdf-parse is 1.1.1, restart Next.js, and upload again.

Metrics or risks are empty

Possible causes:

- Gemini quota is exhausted.
- The document contains no readable text.
- The document contains labels that do not match local fallback patterns.
- The upload was processed by an old server.

Inspect agent logs and re-upload after restarting the server. A valid Gemini key with available quota provides the best results.

Gemini returns 429

This means the project or API key exceeded quota. Wait for quota reset, use a key with available quota, enable billing according to Google terms, or use the local fallback mode.

Gemini returns 503

The provider is temporarily overloaded. The application retries transient failures. If the outage continues, local fallback behavior should keep documents and chat usable.

Research Agent shows a raw provider error

Restart the server so the fallback version of researchAgent.ts is loaded. The fallback should return Local Research Mode with stored evidence.

Production deployment fails at build

Run locally:

```
npm run typecheck
npm run lint
npm run build
```

Fix build errors before deploying. Check Vercel logs for environment and module resolution errors.

13. API Examples

Health check

```
Invoke-WebRequest http://localhost:3000/api/health
```

Seed data

```
Invoke-WebRequest http://localhost:3000/api/seed -Method GET
```

Register a user

```
$body = @{ name = "Demo Analyst"; email = "analyst@example.com"; password =
"password123" } | ConvertTo-Json
Invoke-WebRequest http://localhost:3000/api/auth/register -Method POST -ContentType "application/json" -Body $body
```

API authentication note

Protected endpoints use the NextAuth browser session cookie. For scripts, first authenticate through the application or use an approved integration test setup. Do not disable authentication just to simplify local testing.

14. Database Reference

users

Credential accounts, display names, roles, and timestamps.

research_sessions

User-owned research workspaces with status, tags, and timestamps.

companies

Company name, ticker, sector, industry, and description.

documents

Original file metadata, extracted content, status, chunk count, owner, company, and fiscal year.

document_chunks

Searchable text segments with chunk index, section, and estimated page number.

financial_metrics

Structured financial values connected to documents and companies.

risk_flags

Risk classification, severity, evidence, and recommendation.

analysis_reports

Generated reports and report state.

chat_messages

Conversation messages, agent type, citations, and reasoning.

agent_logs

Agent activity, status, details, duration, and timestamps.

15. Financial Glossary

Term	Meaning
------	---------

Revenue	Money earned from products or services.
Gross profit	Revenue minus cost of goods sold.
Gross margin	Gross profit divided by revenue.
Operating income	Profit from normal operations before interest and taxes.
Operating margin	Operating income divided by revenue.
Net income	Profit after all expenses and taxes.
Net margin	Net income divided by revenue.
EBITDA	Earnings before interest, taxes, depreciation, and amortization.
EPS	Earnings per share.
Current ratio	Current assets divided by current liabilities.
Debt-to-equity	Debt divided by shareholders' equity.

ROE	Net income divided by shareholders' equity.
ROA	Net income divided by total assets.
Operating cash flow	Cash generated by normal operations.
Capital expenditures	Cash spent on long-term assets.
Free cash flow	Operating cash flow minus capital expenditures.
10-K	Annual SEC filing by a public company.
10-Q	Quarterly SEC filing by a public company.
Risk flag	A structured warning or concern found in a document.

16. Recommended Demo Presentation Flow

For a project demonstration:

1. Start PostgreSQL.
2. Start the Next.js app.
3. Open /api/health to prove database connectivity.
4. Sign in with the demo account.
5. Show the dashboard and seeded companies.
6. Open a company and explain metrics.
7. Open Risk Analysis and explain evidence.

8. Open Benchmarking and compare two companies.
9. Open Reports and show report generation.
10. Open Research Agent and ask a cited question.
11. Upload a searchable PDF and show the processing pipeline.
12. Explain Gemini fallback behavior and quota limitations.

17. Final Verification Checklist

Before handing the project to another person:

- ☐ PostgreSQL is running.
- ☐ .env exists locally and is not committed.
- ☐ DATABASE_URL points to the intended database.
- ☐ NEXTAUTH_SECRET is long and private.
- ☐ NEXTAUTH_URL matches the current URL.
- ☐ Gemini key is valid or fallback mode is understood.
- ☐ Schema is applied.
- ☐ Demo data is seeded.
- ☐ Login works.
- ☐ Registration works.
- ☐ Health endpoint returns database connected.
- ☐ Document upload works.
- ☐ Searchable PDF text is visible.
- ☐ DOCX text is visible.
- ☐ Metrics and risks are present or fallback status is understood.
- ☐ Research Agent returns an answer.
- ☐ npm run typecheck passes.
- ☐ npm run lint passes.
- ☐ npm run build passes.
- ☐ Production secrets are separate from local secrets.
- ☐ Production database is backed up.

18. Project Summary

FinResearch AI combines Next.js, PostgreSQL, Drizzle ORM, NextAuth, document parsing, Gemini analysis, structured financial data, and evidence-based research workflows in one application.

The safest operational model is:

1. Make document text extraction reliable.
2. Store and index source evidence.

3. Treat AI enrichment as an optional layer.
4. Preserve usable local fallback behavior when provider quotas or availability change.
5. Validate every important financial value against the original document.
6. Keep all production secrets private and rotate exposed keys.

End of documentation.